

Name or Value? Cue Conflict Reveals When Chain of Thought Overrides Lexical Priors

Anonymous ACL submission

Abstract

Language models are misled by variable names that contradict a program’s behavior, but an accuracy drop cannot show where inside the model the name interferes. Interpretability studies find that intermediate values are computed during chain of thought, yet in their clean tasks the variable name and the computed value never disagree. We introduce a cue-conflict test for multi-step arithmetic in which each problem appears with congruent, neutral, and incongruent variable names under identical arithmetic, and evaluate two Llama models across four difficulty levels and three demonstration sets. Without a chain of thought the models read a variable’s value off its name: a congruent name raises accuracy by up to $\langle\langle\text{facilitationmax}\rangle\rangle$ points and an incongruent one yields the name’s value as the answer up to $\langle\langle\text{lureexcessmax}\rangle\rangle$ points above chance. With a chain of thought the name’s value has no measurable effect on the answer at any level. Linear probes trained on neutral problems show that the lure is decodable only at the name’s own tokens and is erased at the step that writes the value; activation patching localises the no-chain lure to the early-layer representation of the equation that defines the name. Chain of thought thus overrides the lexical prior by replacing it, not by suppressing it. What a chain does not remove is a word-class effect on equation selection, and every effect we measure is smaller than the effect of which three demonstrations are shown.

1 Introduction

Models read code and mathematics in which names carry meaning, and names that contradict the computation reduce accuracy: on real programs, removing or corrupting identifiers degrades execution prediction (Le et al., 2025), misleading comments and hints cut output prediction by a fifth (Lam et al., 2025), and adversarial identifier renaming impairs comprehension through se-

mantic displacement and interference in a prior human–model study (Le et al., 2026). An accuracy drop, however, is compatible with three different failures: a corrupted input representation, a corrupted computation, or a correct computation misread at output, and these differ in whether a written chain of thought can be trusted and in how the failure might be fixed.

Interpretability work on clean tasks cannot separate them either. Kudo et al. (2026) show, with linear probes and activation patching on controlled multi-step arithmetic, that the values of intermediate variables become linearly decodable only as the chain of thought is written, and that the final answer depends causally on the chain. But in their tasks the variable names are letters. The decoded value has exactly one possible source, so nothing in the design can tell “the model computed 5” from “the model read 5 off a cue,” because no cue ever disagrees.

Cue conflict resolves this. In the Stroop task (Stroop, 1935), the word RED printed in blue ink reveals whether reading overrides colour naming; in vision, shape–texture conflict images showed that apparent shape recognition in CNNs was partly texture matching (Geirhos et al., 2019). We bring the design to arithmetic reasoning. Every problem appears in three matched versions with identical arithmetic (Figure 1): neutral names (pen=1 + cup, cup=2 + 3; pen=?), congruent names, where a number word equals the variable’s true value (five=2 + 3), and incongruent names, where it does not (two=2 + 3, true value 5, lure 2). Each version is solved with and without a written chain of thought, and linear probes trained on neutral problems only read out, at every layer and reasoning step, whether the representation holds the true value or the lure.

Our main finding is that the chain overrides the name by replacement: without a chain of thought the models use the name’s value as a shortcut

```

pen=1 + cup, cup=2 + 3; pen=?
pen=1 + five, five=2 + 3; pen=?
pen=1 + two, two=2 + 3; pen=?
pen=1 + two, two=2 + 3, two=5, pen=1 + two,
pen=1 + 5, pen=6
[margin sketch:  $\log p(\text{true}) - \log p(\text{lure})$  across P2-P5,
to be drawn from a real instance once probes exist]

```

Figure 1: One problem in its three matched versions (top), the chain of thought with probe positions P2–P5 (middle), and the probe margin across reasoning steps (bottom). The incongruent name two has true value 5 and lure value 2.

in both directions, with a chain the value of the name is irrelevant to the answer, and inside the model the lure is present only where the name sits and disappears at the step that computes the value. Concretely, we contribute (i) a cue-conflict extension of a controlled arithmetic benchmark with matched twins, generation rules that make the lure never correct for another reason, and the demonstration set as a reported factor; (ii) a layer, position and reasoning-step map of lure versus true value, with and without chain of thought, on two models and four levels; and (iii) causal evidence from activation patching with two control patches on where the no-chain lure lives.

2 Setup

Task. We reuse the generator, difficulty levels and prompt format of Kudo et al. (2026): problems are lists of assignments over single digits with + and −, all values stay in 0..9, and three fixed same-level demonstrations precede each problem. Our main level is their Level 3 (two steps, one forward reference; $v1=d \pm v2$, $v2=d \pm d$; $v1=?$), with Levels 2 and 5 as robustness checks and Level 4 for a control described below. Probes are trained on 10,000 neutral problems; evaluation uses 2,000 matched sets per level with the probe-training digit expressions held out.

Conditions. Each arithmetic problem yields a matched set. The *letter* condition is Kudo et al.’s format and anchors our replication. *Neutral* names are common single-token words. In the *congruent* and *incongruent* conditions exactly one variable, the target, is renamed to a number word that equals or does not equal its true value. The target is either the intermediate variable or the queried one. An *irrelevant-lure* control at Level 4 gives the number-word name to a distractor outside the

queried chain, separating the lexical presence of a number from a binding conflict.

Generation rules. Exactly one variable per instance carries a number word. The lure differs from every variable value and from every operand digit in the problem, so it is never correct for another reason. Neutral words are common concrete nouns that are a single token in every context of the template under each model’s tokenizer (Appendix A), so twins tokenize to identical layouts and length is not a confound. Every incongruent instance has a neutral and a congruent twin with identical arithmetic, and all comparisons are within these sets.

Regimes. In the *chain-of-thought* regime the model writes each step in Kudo et al.’s format (cup=5, pen=1 + 5, pen=6); note that the chain writes the name next to the computed value, which may help the model recover. In the *direct* regime it writes only pen=6. The contrast between regimes is the center of the paper.

Probe positions. P1, the name token two at its definition, decodes the lure trivially and is a reference only. P2 is the last operand of the definition, the first position where the true value can be computed; P3 the query position before any chain; P4 the chain position where the target’s value is about to be written; P5 the final answer position. All claims about lure interference are made at P2–P5.

Probes and patching. Probes are single linear layers trained with Kudo et al.’s recipe on the neutral condition only, then evaluated on congruent and incongruent instances; training on incongruent data would let the probe learn the lure. We report accuracy for the true value, the *lure mass* (probability on the lure class), and the *margin* $\log p(\text{true}) - \log p(\text{lure})$; the *crossover* is the first step at which the margin becomes and stays positive. Selectivity against a control task (Hewitt and Liang, 2019) accompanies every probe, over three seeds. For causation we patch residual-stream activations at the misleading name’s token positions from the matched neutral twin into the incongruent run (Zhang and Nanda, 2024), sweep layers, and measure the *recovery rate*, the share of lure errors that become correct. A patch from a different neutral word measures patching damage; a patch from a different lure tests whether the answer follows it.

Model	Regime	Target	Neutral	Congr.	Incongr.	Lure rate	Interf.	Facil.
Llama-3.2-3B	cot	v2	96.6 (95.8, 97.4)	99.5 (99.1, 99.8)	99.4 (99.0, 99.7)	0.0 (0.0, 0.0)	-2.8 (-3.6, -1.9)	2.9 (2.0, 3.7)
Llama-3.2-3B	cot	v1	96.6 (95.8, 97.4)	95.1 (94.2, 96.0)	94.6 (93.6, 95.5)	0.4 (0.2, 0.8)	2.0 (1.1, 2.9)	-1.5 (-2.5, -0.5)
Llama-3.2-3B	direct	v2	13.4 (11.8, 14.8)	17.9 (16.2, 19.6)	13.8 (12.2, 15.2)	8.1 (6.8, 9.3)	-0.4 (-1.6, 0.8)	4.6 (3.3, 5.9)
Llama-3.2-3B	direct	v1	13.4 (11.8, 14.8)	18.9 (17.2, 20.6)	13.9 (12.5, 15.4)	8.6 (7.4, 10.0)	-0.5 (-1.7, 0.6)	5.5 (4.3, 6.9)
Llama-3.1-8B	cot	v2	85.7 (84.2, 87.2)	97.5 (96.8, 98.2)	84.0 (82.3, 85.5)	0.9 (0.5, 1.4)	1.7 (0.2, 3.2)	11.8 (10.2, 13.5)
Llama-3.1-8B	cot	v1	85.7 (84.2, 87.2)	82.5 (80.9, 84.1)	83.6 (82.0, 85.3)	0.7 (0.3, 1.0)	2.1 (0.8, 3.2)	-3.2 (-4.5, -1.9)
Llama-3.1-8B	direct	v2	41.3 (39.1, 43.5)	47.9 (45.6, 50.1)	45.8 (43.7, 48.0)	3.5 (2.7, 4.2)	-4.5 (-5.8, -3.1)	6.6 (5.2, 8.0)
Llama-3.1-8B	direct	v1	41.3 (39.1, 43.5)	48.8 (46.7, 51.0)	44.4 (42.3, 46.6)	3.5 (2.7, 4.2)	-3.1 (-4.5, -1.7)	7.5 (5.9, 9.0)

Table 1: Accuracy (%), lure rate, interference and facilitation per model, regime and target position, Level 3, pooled over three demonstration sets with 95% cluster-bootstrap intervals over matched sets.

Models. `<<model-list>>`, base models of 3B and 8B parameters from Kudo et al.’s panel, plus `<<extra-models>>`. Behavioural metrics are accuracy, lure rate (answers equal to the lure), *interference* (neutral minus incongruent accuracy) and *facilitation* (congruent minus neutral), with cluster-bootstrap confidence intervals over matched sets.

3 Results

Replication. On the letter condition our per-token probes reproduce Kudo et al.’s Table 3 for Llama-3.2-3B exactly: the queried variable becomes decodable at equation 5 of the chain and the intermediate at equation 2, with pre-chain maxima of `<<replication-pre-v1>>` and `<<replication-pre-v2>>` (theirs: 17.8 and 33.2).

RQ1: behaviour. Table 1 reports, per model and regime, accuracy and the pre-registered contrasts pooled over three demonstration sets, with a contrast claimed only when its sign holds in every set. Without a chain the name’s value is used as a shortcut: a congruent name on the queried variable adds +4.8 points for Llama-3.1-8B at level 3 and up to `<<facilitationmax>>` points at level 5, and an incongruent name yields lure answers `<<lureexcessllama318bdirectv1>>` points above chance at level 3 and up to `<<lureexcessmax>>` points where accuracy is at floor. With a chain the same contrasts are within `<<chain-lure-bound>>` points of zero in every cell, and congruent and incongruent names move together. The irrelevant-lure control at level 4 separates presence from binding: a number word on the distractor costs accuracy but produces no lure answers. What the chain does not remove is a word-class effect: a number-word name on the queried variable at level 4 costs Llama-3.2-3B +1.1 points whether or not its value agrees, and the error analysis shows the model restating the wrong equation next.

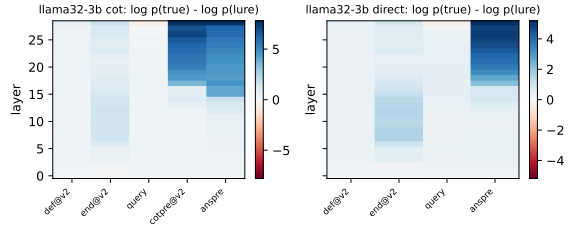


Figure 2: Probe margin $\log p(\text{true}) - \log p(\text{lure})$ by layer and position, incongruent condition, chain of thought (left) and direct answer (right), Llama-3.2-3B. Positive means the true value dominates.

RQ2: representation. Figure 2 maps the probe margin over layers and positions P2–P5 in the incongruent condition, chain of thought beside direct answer. At P2, the end of the defining equation, the true value is already partly computed (accuracy `<<p2-acc>>`, selectivity `<<p2-sel1>>`) and the lure is weak (lure rate `<<p2-lure>>`). At P3, the query, a neutral-trained probe reads the lure in `<<p3-lure>>` of instances, but the control task scores `<<p3-ctl1>>` there: the readout is the name’s identity, not a value, and we do not claim it. At P4 and P5 the true value is decodable at `<<p4-acc>>` with the lure at zero in every condition and level; per-token probes show the lure decodable only at the name’s own tokens, and a number word bound to a computed variable has lure mass `<<bound-mass>>` at the answer against `<<unbound-mass>>` for an unbound one on the distractor. The outcome cell is “lure fades, answer correct”, reached by replacement at the value step rather than suppression.

RQ3: causation. Figure 3 plots, for the direct regime, the share of lure errors removed by patching the name’s tokens from the matched neutral twin, by layer, with the two controls. For Llama-3.2-3B the neutral patch removes `<<lure-removed-3b>>` of lure errors and restores the true-minus-lure margin fully (normalised logit difference `<<nld-3b>>`), the alternative-lure patch is followed in `<<follow-new-lure>>` of cases, and the span grid places the effect in the defining equation at layers `<<grid-layers>>`. The word control changes `<<control-damage>>` of correct answers at this floor accuracy, so we report the localisation and not a recovery rate. For Llama-3.1-8B the no-chain lure rate at level 3 is at chance and both patches behave alike; the effect exists only where the lure does. With a chain there are no lure errors

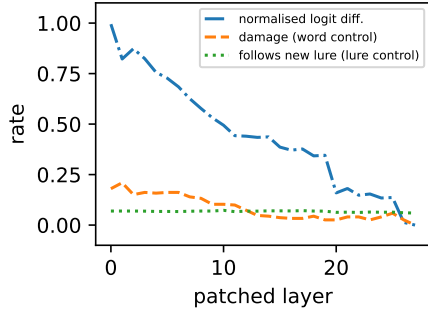


Figure 3: Lure errors removed by patching the name’s tokens from the neutral twin, by layer, direct regime, with the neutral-word control patch (damage) and the alternative-lure patch.

to patch and both controls are clean.

4 Related Work

Kudo et al. (2026) established when sub-answers are computed during chain of thought on clean arithmetic; we add cue conflict to ask what that computation is sensitive to. Two recent studies bring the Stroop design to language models: Wang (2026) pits a local redefinition of a word against its familiar associate and restores the lost margin by patching from a matched neutral prompt; Hu et al. (2026) read a congruency effect off next-token probabilities in a colour-rule task every model solves. Both measure one next token; neither has errors, a chain of thought, or probes over reasoning steps. Faithfulness has been measured from the outside by perturbing prompts, chains or parameters (Turpin et al., 2023; Lanham et al., 2023; Chen et al., 2025; Tutek et al., 2025); we measure it inside the model under a cue whose true and lure values are both known. On arithmetic, Liu (2026) show that small models copy the trailing chain number and Prabhakar et al. (2024) that an output prior can override a correct chain; our late-readout outcome is stated against a positional-copy control. Misleading identifiers and comments hurt on real code (Le et al., 2026, 2025; Lam et al., 2025); we trade realism for localisation. Binding work asks how a value attaches to a name (Feng and Steinhardt, 2024; Wu et al., 2025; Gur-Arieh et al., 2026); we ask whether the attachment survives a name carrying a competing value. Shi et al. (2023) add irrelevant context; here the distractor is the name itself. The design is Stroop (1935) and Geirhos et al. (2019).

5 Conclusion

A misleading variable name is used when the model has nothing better: without a chain of thought the name’s value becomes the answer, with a chain it is replaced at the step that computes the value and never reaches the output. For faithfulness this is the reassuring cell of the outcome table, and it comes with two cautions: the chain leaves a word-class effect on which equation the model attends to next, and the demonstration set moves accuracy more than any name.

Limitations

The task is synthetic arithmetic with short chains, so the results need not transfer directly to real code, which is the motivation and not the finding. Models are at most 8B parameters; within-family comparisons give the direction of the trend only. Lures are hand-picked English number words; digit-bearing names such as x2, other languages and optimised lures are future work. Linear probes can miss nonlinearly encoded information; the patching experiment partly addresses this, but hidden states carry mixed information and patching cannot rule out side effects entirely, which is why two control patches are reported. The chain-of-thought format writes the name beside the computed value at every step, which may itself aid recovery; the direct regime bounds that effect from the other side.

References

- Yanda Chen et al. 2025. Reasoning models don’t always say what they think. *arXiv preprint arXiv:2505.05410*.
- Jiahai Feng and Jacob Steinhardt. 2024. How do language models bind entities in context? In *International Conference on Learning Representations*.
- Robert Geirhos, Patricia Rubisch, Claudio Michaelis, Matthias Bethge, Felix A. Wichmann, and Wieland Brendel. 2019. ImageNet-trained CNNs are biased towards texture; increasing shape bias improves accuracy and robustness. In *International Conference on Learning Representations*.
- Yoav Gur-Arieh, Mor Geva, and Atticus Geiger. 2026. Mixing mechanisms: How language models retrieve bound entities in-context. In *International Conference on Learning Representations*. ArXiv:2510.06182.
- John Hewitt and Percy Liang. 2019. Designing and interpreting probes with control tasks. In *Proceedings of the 2019 Conference on Empirical Methods*

331	<i>in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP).</i>	389
332		390
333		391
334	Xiaoyang Hu, Mike Angstadt, Shane Storks, Zan	392
335	Huang, Aman Taxali, Alex Weigard, Richard L.	393
336	Lewis, and Chandra Sripada. 2026. Conflict and	394
337	congruency effects in large language models: In-	395
338	weight and in-context competition in a verbal con-	396
339	flict task. <i>arXiv preprint arXiv:2608.11510</i> .	397
340	Keito Kudo, Yoichi Aoki, Tatsuki Kuribayashi,	398
341	Shusaku Sone, Masaya Taniguchi, Ana Brassard,	399
342	Keisuke Sakaguchi, and Kentaro Inui. 2026. LLMs	400
343	faithfully and iteratively compute answers during	401
344	CoT: A systematic analysis with multi-step arith-	402
345	metics. In <i>Findings of the Association for Com-</i>	403
346	<i>putational Linguistics: EACL 2026</i> , pages 1114–	
347	1153. Code: https://github.com/keitokudo/	
348	faithful-cot-multistep-arithmetic .	
349	Man Ho Lam, Chaozheng Wang, Jen-tse Huang, and	
350	Michael R. Lyu. 2025. CodeCrash: Exposing LLM	
351	fragility to misleading natural language in code re-	
352	asoning. In <i>Advances in Neural Information Process-</i>	
353	<i>ing Systems</i> , volume 38. ArXiv:2504.14119.	
354	Tamera Lanham, Anna Chen, Ansh Radhakrishnan,	
355	Benoit Steiner, Carson Denison, Danny Hernan-	
356	dez, Dustin Li, Esin Durmus, Evan Hubinger, Jack-	
357	son Kernion, Kamilė Lukošūtė, Karina Nguyen,	
358	Newton Cheng, Nicholas Joseph, Nicholas Schiefer,	
359	Oliver Rausch, Robin Larson, Sam McCandlish,	
360	Sandipan Kundu, and 11 others. 2023. Measuring	
361	faithfulness in chain-of-thought reasoning. <i>arXiv</i>	
362	<i>preprint arXiv:2307.13702</i> .	
363	Cuong Chi Le, Minh V. T. Pham, Cuong Duc Van,	
364	Hoang N. Phan, Huy N. Phan, and Tien N. Nguyen.	
365	2025. When names disappear: Revealing what	
366	LLMs actually understand about code. <i>arXiv</i>	
367	<i>preprint arXiv:2510.03178</i> .	
368	Jack Le, Anh H. N. Nguyen, and Tien N. Nguyen. 2026.	
369	Do machines struggle where humans do? LLM and	
370	human comprehension of obfuscated code. <i>arXiv</i>	
371	<i>preprint arXiv:2606.31725</i> . Cite in the third person	
372	under [review].	
373	Ming Liu. 2026. The readout shortcut: Positional	
374	number copying dominates arithmetic CoT read-	
375	out in small language models. <i>arXiv preprint</i>	
376	<i>arXiv:2605.22870</i> .	
377	Akshara Prabhakar, Thomas L. Griffiths, and	
378	R. Thomas McCoy. 2024. Deciphering the	
379	factors influencing the efficacy of chain-of-thought:	
380	Probability, memorization, and noisy reasoning.	
381	<i>arXiv preprint arXiv:2407.01687</i> . Venue to verify	
382	(likely Findings of EMNLP 2024).	
383	Freda Shi, Xinyun Chen, Kanishka Misra, Nathan	
384	Scales, David Dohan, Ed H. Chi, Nathanael Schärli,	
385	and Denny Zhou. 2023. Large language models can	
386	be easily distracted by irrelevant context. In <i>Pro-</i>	
387	<i>ceedings of the 40th International Conference on</i>	
388	<i>Machine Learning</i> .	
	J. Ridley Stroop. 1935. Studies of interference in serial	389
	verbal reactions. <i>Journal of Experimental Psychol-</i>	390
	<i>ogy</i> , 18(6):643–662.	391
	Miles Turpin, Julian Michael, Ethan Perez, and Samuel	392
	Bowman. 2023. Language models don’t always say	393
	what they think: Unfaithful explanations in chain-of-	394
	thought prompting. In <i>Advances in Neural Informa-</i>	395
	<i>tion Processing Systems</i> , volume 36, pages 74952–	396
	74965.	397
	Martin Tutek, Fateme Hashemi Chaleshtori, Ana	398
	Marasović, and Yonatan Belinkov. 2025. Measuring	399
	chain of thought faithfulness by unlearning reason-	400
	ing steps. In <i>Proceedings of the 2025 Conference</i>	401
	<i>on Empirical Methods in Natural Language Process-</i>	402
	<i>ing</i> .	403
	Han-yu Wang. 2026. Persistent priors, preserved tar-	404
	gets: A Stroop-style paradigm for lexical override.	405
	<i>arXiv preprint arXiv:2606.07555</i> .	406
	Yiwei Wu, Atticus Geiger, and Raphaël Millièvre. 2025.	407
	How do transformers learn variable binding in sym-	408
	bolic programs? In <i>Proceedings of the 42nd Inter-</i>	409
	<i>national Conference on Machine Learning</i> .	410
	Fred Zhang and Neel Nanda. 2024. Towards best prac-	411
	tices of activation patching in language models: Met-	412
	rics and methods. In <i>International Conference on</i>	413
	<i>Learning Representations</i> .	414
	A Tokenizer checks	415
	Per-model lists of neutral words that are one to-	416
	ken in every context (line start, after a comma, as	417
	an operand), and the token layout of the template:	418
	<code><<appendix-tokenizer>></code> .	419
	B Full layer sweeps and selectivity	420
	<code><<appendix-sweeps>></code>	421
	C Additional models and levels	422
	<code><<appendix-models>></code>	423